

GUIA OPERACIONAL

Implementação e manutenção do Domínio Maestro

Procedimentos para desenvolvimento, homologação, produção, segurança e recuperação

AMBIENTE REAL	LOCAL DE TESTE	BANCO DE PRODUÇÃO
dominiomaestro.com.br KVM2 / maestro-web	127.0.0.1:4173 build de produção	Maestro BD producao São Paulo / sa-east-1

Documento atualizado em 06/09/2026

1. Objetivo e visão geral

Este documento explica como implementar alterações no Domínio Maestro e como manter o sistema depois da publicação. O fluxo separa código, banco, funções, ambiente local, homologação e produção para reduzir o risco de uma alteração de teste afetar a equipe.

Regra principal

Toda alteração deve ser testada primeiro em ambiente isolado. O domínio real usa o projeto Supabase de produção e só recebe um build que já passou pela validação local e pela revisão do lote funcional.

Mapa rápido do documento

Seção	Use quando
2. Ambientes e acessos	Precisar saber qual sistema ou painel deve ser acessado.
3. Fluxo de implementação	For iniciar uma alteração e organizar branch, testes e commit.
4. Configuração local	For rodar o app ou preparar um build.
5. Supabase	For alterar dados, RLS, migrations ou Edge Functions.
6. Publicação no KVM2	For colocar uma versão aprovada no domínio real.
7. Manutenção e rollback	For fazer rotina, investigar erro ou recuperar uma versão.
8. Checklist final	Antes de considerar uma entrega concluída.

Estado atual confirmado

- O banco de produção contém 83.808 registros migrados, 13 colaboradores, 108 projetos e 1.056 jobs.
- O login oficial por Edge Function foi validado no localhost e no domínio real.
- Dashboard, Projetos, Jobs e Propostas foram carregados no domínio real com os dados migrados.
- O site é servido pelo container maestro-web no KVM2; um backup do conteúdo anterior foi mantido antes da publicação.

2. Ambientes e acessos

Use sempre o ambiente correspondente ao tipo de ação. Produção é para dados reais e não deve ser usada para ensaiar exclusões, mudanças de senha ou testes destrutivos.

Ambiente	Identificação	Uso
Código	GitHub: Seja-Dominio/dominio-maestro	Branches, revisão e histórico do código.
Local	http://127.0.0.1:4173	Validação do build apontado ao Supabase de produção, sem publicar no domínio.
Homologação	Projeto Supabase separado (a criar)	Testes com dados isolados. Nunca apontar para produção.
Produção	Maestro BD producao / fwpisypieezjhtqxlmqv	Banco real em São Paulo, região sa-east-1.
Site real	https://dominiomaestro.com.br	Frontend servido pelo KVM2 e container maestro-web.

Acessos necessários

- GitHub: acesso ao repositório e à branch de trabalho.
- Supabase: acesso ao projeto correto e permissão adequada para Database, Functions, Storage e Secrets.

- KVM2: chave SSH configurada localmente; nunca colar a chave em código, PDF, chat ou GitHub.
- Terminal local: Node.js, npm, Supabase CLI quando necessário e dependências instaladas.

Não confundir projetos

O projeto legado tqmfuskvllpqmvayjuqu é a origem histórica. O destino atual de produção é fwpisypiiezjhtqxlmqv. Confirme o ref antes de aplicar qualquer SQL ou publicar uma função.

3. Fluxo padrão de implementação

Organize cada entrega como um lote funcional coerente. Um lote pode conter, por exemplo, propostas, permissões, agenda ou dashboard, desde que o escopo seja testável como uma unidade.

Etapa	Ação	Critério de saída
1	Atualizar a branch	Partir da branch de trabalho e confirmar o estado do Git antes de editar.
2	Implementar o lote	Alterar o frontend, API, migrations ou Functions necessárias. Reutilizar padrões existentes.
3	Validar localmente	Rodar o app em localhost e testar leitura, criação, edição, filtros, permissões e mensagens de erro.
4	Validar em homologação	Usar um Supabase separado ou dados isolados. Nunca experimentar alterações destrutivas em produção.
5	Rodar os gates	Executar build, testes disponíveis e revisão do diff. Registrar o que foi validado.
6	Criar o commit do lote	Um commit funcional por lote; mensagem objetiva e rastreável.
7	Fazer um único push	Enviar a branch somente quando o lote estiver validado e pronto para revisão.
8	Publicar produção	Após aprovação, publicar Functions e frontend no destino correto e verificar o domínio.
9	Integrar a branch principal	Fazer merge somente depois da validação externa e da janela de observação.

Comandos de início

```
git status
git switch -c codex/nome-do-lote
npm install
npm run dev
```

Antes de usar um nome de branch diferente, confirme o padrão do repositório. Não apague alterações existentes de outro trabalho.

4. Configuração e execução local

Os arquivos de ambiente locais são ignorados pelo Git. Eles devem conter somente valores necessários para a máquina ou para o servidor e nunca devem ser enviados ao repositório.

4.1 Ambiente de produção local

Para reproduzir o build que está no domínio, use o arquivo local de produção e a porta autorizada pelas Edge Functions:

```
npm install
npm run build
npm run preview -- --host 127.0.0.1 --port 4173
```

Abra <http://127.0.0.1:4173>. O arquivo `.env.production.local` deve apontar para <https://fwpisypiiezjhtqxlmqv.supabase.co> e usar somente a chave pública do frontend.

4.2 Ambiente de homologação

A homologação real ainda depende da criação de um projeto Supabase separado. Depois de criado, preencha o arquivo ignorado `.env.test` com a URL e a chave pública desse projeto.

```
cp config/env.test.example .env.test
npm run dev:test
# abrir http://127.0.0.1:4174
npm run build:test
```

Bloqueio de segurança

O modo test deve rejeitar as URLs conhecidas de produção. Se `.env.test` apontar para `fwpisypieezjhtqxlmqv` ou `tqmfuskvllpqmvayjuqu`, pare e corrija antes de fazer login ou gravar dados.

4.3 Gates locais

```
npm run build
npm run lint
npm run typecheck
```

O build atual passa. O lint ainda possui débitos históricos, incluindo imports não utilizados e regras ausentes em artefatos gerados; não trate um lint vermelho como aprovação automática. Registre o resultado e separe correção de lint do lote funcional.

5. Supabase: banco, segurança e funções

O Supabase concentra o banco, autenticação compatível, Edge Functions, Storage e parte da segurança. Toda operação remota deve ser feita no projeto correto e verificada depois.

5.1 Banco de dados

- Use migrations versionadas para mudanças de schema. Dê nomes descritivos e mantenha uma forma de rollback.
- Para leitura e diagnóstico, confirme tabelas, colunas, índices, constraints, RLS e policies antes de editar.
- Tabelas expostas pelo PostgREST precisam de RLS ativado e policies mínimas para cada operação necessária.
- Nunca coloque service-role key no frontend. Ela bypassa RLS e deve ficar somente em ambiente de servidor ou função protegida.
- Para migração de dados, compare contagem total, entidades, IDs e datas antes e depois. A origem deve permanecer intacta.

5.2 Consultas de verificação

```
select count(*) from public.legacy_records;
select entity, count(*)
from public.legacy_records
group by entity order by entity;

select tablename, rowsecurity
from pg_tables
where schemaname = 'public';
```

Se a consulta retornar uma tabela pública sem RLS, trate como alerta de segurança e crie a policy adequada antes de expor a tabela ao frontend.

5.3 Edge Functions

- O login collaborator-login usa uma sessão própria assinada e CORS para localhost 4173/4174/4175 e o domínio real.
- Ao publicar uma função, confirme nome, projeto, versão, configuração de JWT e secrets usados pela função.
- Teste OPTIONS/CORS, resposta de sucesso, credencial inválida e acesso a dados autenticado.

- Depois do deploy, consulte invocações e logs; não considere o upload concluído apenas porque a publicação terminou.

```
supabase functions list --project-ref fwpisypieezjhtqxlmqv
supabase functions deploy collaborator-login --project-ref fwpisypieezjhtqxlmqv
```

Se o CLI pedir autenticação, use uma sessão OAuth ou access token válido sem colar o valor no código. Se o conector estiver sem escopo de Functions, autorize `edge_functions:write` ou publique pelo editor autenticado do painel.

5.4 Storage

O bucket `job-attachments` deve permanecer privado. Upload e download devem passar pela função autorizada, com validação de sessão e nome/tipo/tamanho do arquivo.

6. Publicação no KVM2

O KVM2 serve o conteúdo compilado de dist pelo container `maestro-web`. O código-fonte fica no GitHub e os dados/funções ficam no Supabase.

6.1 Preparar e testar o build

```
npm run build
curl -I http://127.0.0.1:4173/
```

Verifique título Domínio Maestro, carregamento do JavaScript e referência do projeto `fwpisypieezjhtqxlmqv` no bundle. Faça login e percorra Dashboard, Projetos, Jobs e Propostas.

6.2 Backup e envio

```
# Substitua DATA-HORA por um identificador único
ssh -i /Users/grimm/.ssh/maestro-kvm2 root@187.127.27.151 \
'docker cp maestro-web:/usr/share/nginx/html /tmp/maestro-web-backup-DATA-HORA'

scp -i /Users/grimm/.ssh/maestro-kvm2 -r dist \
root@187.127.27.151:/tmp/maestro-dist-DATA-HORA

ssh -i /Users/grimm/.ssh/maestro-kvm2 root@187.127.27.151 \
'docker cp /tmp/maestro-dist-DATA-HORA/. maestro-web:/usr/share/nginx/html/'
```

6.3 Verificação externa

```
curl -I https://dominiomaestro.com.br/
ssh -i /Users/grimm/.ssh/maestro-kvm2 root@187.127.27.151 \
'docker ps --format "{{.Names}}\t{{.Status}}" | grep '^maestro-web'
```

Esperado: HTTP 200, container `maestro-web` ativo, login funcionando e telas principais carregando dados. Se o navegador usar cache, faça uma atualização completa ou abra uma janela privada para a verificação.

Nunca enviar segredos

Não copie `.env.local`, `.env.production.local`, `service-role key`, `access token`, senha de banco ou chave SSH para o KVM2 junto com dist. O frontend deve conter somente valores públicos e necessários.

7. Manutenção, diagnóstico e rollback

7.1 Rotina recomendada

Periodicidade	Verificação
A cada lote	Build, login, leitura/gravação do escopo, console do navegador e diff.
Antes de produção	Backup do frontend, contagem do banco, status das Functions e RLS/policies.
Semanal	Erros das Functions, falhas de autenticação, espaço do servidor, HTTPS e backups.
Mensal	Atualizar dependências com revisão, revisar secrets, permissões de equipe e advisors do Supabase.

7.2 Diagnóstico por sintoma

Sintoma	Primeira verificação	Correção provável
Tela branca	Console e resposta do bundle; confirme URL do Supabase.	Corrigir erro de build/import e publicar novamente.
Login falha	CORS, status da collaborator-login e logs.	Corrigir origem permitida, secret ou versão da Function.
Dashboard vazio	maestro-data, sessão e contagem em legacy_records.	Corrigir CORS/sessão ou função; não recarregar dados sem comparar.
Projeto sem jobs	IDs project_id, entidade Job e resposta da API.	Corrigir vínculo/filtro; preservar IDs migrados.
HTTPS offline	curl -I, certificado, nginx e container.	Corrigir proxy/certificado; o DNS sozinho não publica o app.
RLS advisor	Tabela, rowsecurity e policies.	Ativar RLS e criar policies mínimas e testadas.

7.3 Rollback do frontend

```
# Restaurar o backup do conteúdo anterior
ssh -i /Users/grimm/.ssh/maestro-kvm2 root@187.127.27.151 \
'docker cp /tmp/maestro-web-backup-DATA-HORA/. maestro-web:/usr/share/nginx/html/'

curl -I https://dominiomaestro.com.br/
```

Para rollback de banco, use uma migration reversível ou restauração oficial do Supabase. Não execute DROP, DELETE em massa ou alteração de credenciais como tentativa de rollback sem uma confirmação explícita e uma cópia verificável.

8. Checklist de entrega

- [] O lote funcional está descrito e limitado.
- [] A branch de trabalho foi confirmada e não há alterações alheias incluídas.
- [] O teste local foi feito na origem correta.
- [] A homologação usou base separada ou dados isolados.
- [] Build passou e o resultado foi aberto no navegador.
- [] Login, permissões e mensagens de erro foram testados.
- [] Leitura, criação, edição, filtros e download foram testados quando aplicável.
- [] RLS e policies foram revisados para tabelas públicas.
- [] Functions foram publicadas no ref correto e seus logs foram conferidos.
- [] Backup do frontend foi criado antes do KVM2.
- [] O domínio respondeu HTTP 200 e carregou as telas principais.
- [] Foi feito um commit por lote funcional e um único push ao concluir o lote.
- [] O merge para a branch principal aguarda a validação externa final.

9. Pendências do ambiente atual

Pendência	Próxima ação
Homologação isolada	Criar projeto Supabase separado, aplicar schema/functions e preencher .env.test.
GitHub	Revisar o worktree, separar commits por lote e enviar a branch correta.
Resumo por IA	Migrar ou substituir a chamada legada Base44 que ainda retorna 405.
Lint	Planejar lote separado para imports não utilizados e configuração das regras.

Contato de emergência

Em caso de dúvida, pare antes de alterar produção. Registre o sintoma, URL, horário, tela afetada, status HTTP e log seguro. Nunca registre senhas, tokens ou chaves.

Referências operacionais: docs/ambiente-homologacao.md, config/env.test.example, config/env.maestro-bd-production.example e deploy/.